

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving. (BL5,BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 10

Total Marks: 50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I'M **EDIGA KHADRI NATH GOWD** / 192472296 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: EDIGA KHADRI NATH GOWD

Annexure A

Pseudocode Writing Task (Algorithm Design)

Q106. Subset Generation with Minimum and Maximum Size Bounds

Aim

To generate all possible subsets of a given set whose sizes lie between a specified **minimum size** and **maximum size** using the **Backtracking algorithm**, while avoiding duplicate subsets and pruning invalid branches.

Materials Required

- Computer/Laptop
- Python 3.x
- Python IDLE / VS Code / PyCharm
- Basic knowledge of recursion and backtracking

Procedure

1. Read the input set of elements.
2. Read the minimum subset size MIN.
3. Read the maximum subset size MAX.
4. Start with an empty subset.
5. For every element, make an **include** or **exclude** decision.
6. Track the current subset size.
7. If the subset size exceeds MAX, prune that branch.
8. If the remaining elements cannot satisfy MIN, prune that branch.
9. Store the subset whenever its size is between MIN and MAX.

10. Continue until all valid subsets are generated.

Algorithm / Pseudocode

Algorithm BoundedSubset(A, MIN, MAX)

Input:

A → Set/list of n elements

MIN → Minimum allowed subset size

MAX → Maximum allowed subset size

Output:

All subsets whose size is between MIN and MAX

Procedure Generate(index, currentSubset)

currentSize ← size(currentSubset)

remaining ← n - index

// Pruning condition 1

if currentSize > MAX then

return

// Pruning condition 2

if currentSize + remaining < MIN then

return

```
// Valid subset
```

```
if currentSize >= MIN AND currentSize <= MAX then
```

```
    print currentSubset
```

```
// All elements processed
```

```
if index = n then
```

```
    return
```

```
// Include current element
```

```
add A[index] to currentSubset
```

```
Generate(index + 1, currentSubset)
```

```
// Exclude current element
```

```
remove A[index] from currentSubset
```

```
Generate(index + 1, currentSubset)
```

End Procedure

```
Generate(0, emptySubset)
```

End Algorithm

Implementation

```
def generate_subsets(A, minimum, maximum):
```

```
result = []
```

```
n = len(A)
```

```
def backtrack(index, current):
```

```
    current_size = len(current)
```

```
    remaining = n - index
```

```
    # Pruning
```

```
    if current_size > maximum:
```

```
        return
```

```
    if current_size + remaining < minimum:
```

```
        return
```

```
    # Store valid subset
```

```
    if minimum <= current_size <= maximum:
```

```
        result.append(current.copy())
```

```
    if index == n:
```

```
        return
```

```
    # Include element
```

```
    current.append(A[index])
```

```
    backtrack(index + 1, current)
```

```
# Exclude element
```

```
current.pop()
```

```
backtrack(index + 1, current)
```

```
backtrack(0, [])
```

```
return result
```

```
# Input
```

```
A = [1, 2, 3, 4]
```

```
minimum = 2
```

```
maximum = 3
```

```
subsets = generate_subsets(A, minimum, maximum)
```

```
print("Valid subsets:")
```

```
for subset in subsets:
```

```
    print(subset)
```

Result

```
def generate_subsets(A, minimum, maximum):
    result = []
    n = len(A)

    def backtrack(index, current):
        current_size = len(current)
        remaining = n - index

        # Pruning
        if current_size > maximum:
            return

        if current_size + remaining < minimum:
            return

        # Store valid subset
        if minimum <= current_size <= maximum:
            result.append(current.copy())

        if index == n:
            return

        # Include element
        current.append(A[index])
        backtrack(index + 1, current)

        # Exclude element
        current.pop()
        backtrack(index + 1, current)

    backtrack(0, [])
    return result

# Input
A = [1, 2, 3, 4]
```

Console

```
Run started
Initializing environment
Installing packages
Running code
Valid subsets:
[1, 2]
[1, 2, 3]
[1, 2, 3]
[1, 2]
[1, 2, 4]
[1, 2]
[1, 3]
[1, 3, 4]
[1, 3]
[1, 4]
[2, 3]
[2, 3, 4]
[2, 3]
[2, 4]
[3, 4]
Run completed in 1103.7999999523163ms
```

For the input:

Set = {1, 2, 3, 4}

Minimum size = 2

Maximum size = 3

The algorithm generates all subsets of size 2 and 3 without duplication.

Conclusion

Thus, all subsets satisfying the given minimum and maximum size constraints were generated successfully using **Backtracking with pruning**.